

Universidade Técnica Do Atlântico

Instituto de Engenharias e Ciências do Mar

Sidnei Cruz

scruz@uta.cv



UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO

CAMPUS
DO MAR

Estruturas de Dados

- Programa
 - Um programa pode ser visto como a especificação formal da solução de um problema - **Niklaus Wirth** expressa em sua equação:

programa = algoritmo + estruturas de dados

- **onde:** o algoritmo contém a lógica do programa e os dados são organizados em estruturas de dados.



UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO

CAMPUS
DO MAR

Estruturas de Dados

- Estruturas de Dados
 - A qualidade da solução de um problema depende, entre outros fatores, da forma como estão organizados os dados relevantes
 - Encontrar o número do telefone de um certo assinante no conjunto de todos os assinantes.
- Em geral, os dados devem ser dispostos de forma a tornar **eficientes** o acesso e a modificação dos mesmos pelos algoritmos



Estruturas de Dados

- O que é uma Estrutura de Dados (ED)?
 - Tipos de Dados
 - Tipos de Dados Abstratos (TDA)
- Embora estes termos sejam parecidos, eles têm significados diferentes.
- Em linguagens de programação, o tipo de dados de uma variável define o conjunto de valores que a variável pode assumir
- Por exemplo, uma variável do tipo lógico pode assumir o valor verdadeiro(True) ou falso (False).

Estruturas de Dados

- Um tipo de dados abstratos (TDA) é um modelo matemático acompanhado das operações definidas sobre o modelo
- Um TDA também pode ser considerado um tipo de dados definido pelo utilizador
- Estrutura de Dados é um método particular de se implementar um TDA.



Estruturas de Dados

- A implementação de um TDA escolhe uma ED para representá-lo.
- Cada ED é construída dos tipos primitivos (inteiro, real, char,...) ou dos tipos compostos (vector, estrutura,...) de uma linguagem de programação.
- Exemplos de TDA:

Lineares	Não Lineares
-----------------	---------------------

Listas	Árvores
--------	---------

Pilhas	Grafos
--------	--------

Filas	
-------	--

⋮	
---	--



Estruturas Auto-Referenciadas

- As estruturas auto-referenciadas permitem criar estruturas de dados dinâmicas, utilizando ponteiros:
 - Listas (simplesmente e duplamente ligadas)
 - Árvores
 - Tabelas de dispersão
- Estruturas em que um campo da estrutura é um apontador para outra estrutura do mesmo tipo

```
typedef struct ponto {  
    double x;  
    double y;  
    struct ponto *origem;  
} Ponto;
```



Tabelas

- Colecção de itens
 - Inteiros, reais, caracteres
 - Estruturas
 - Tabelas, Ponteiros
- Guardados em posições consecutivas de memória

```
int tab[N];
```



- Programador é responsável por respeitar limites

Tabelas

- Em C tabelas podem ser
 - De dimensão fixa
 - Alocadas dinamicamente

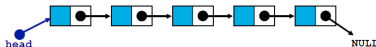
```
#define N 100  
int tab1[N];  
int *tab2 = (int *) malloc(N*sizeof(int));
```

- Acesso alternativo a tabelas

```
x = tab2[i];  
y = *(tab2+i);
```

Tabelas

- Vantagens
 - Manipulação simples
 - Facilidade de acesso ao n -ésimo elemento
- Desvantagens
 - Tamanho limitado
 - Necessidade de realocar e copiar todos os elementos se desejar aumentar dimensão da tabela
 - Desperdício de memória
- Alternativa: Listas



Listas Simplesmente Ligadas

- Conjunto de nós



- Cada nó contém
 - Informação útil
 - Ponteiro para o próximo nó

```
struct node {  
    Item item;  
    struct node *next;  
};
```

Listas Simplesmente Ligadas

- **Vantagens:**
 - Tamanho ilimitado (limite na memória disponível na máquina)
 - Alocação de memória apenas para os elementos que queremos representar
 - Inserção e remoção simples
- **Desvantagens:**
 - Mais difícil o acesso ao n -ésimo elemento

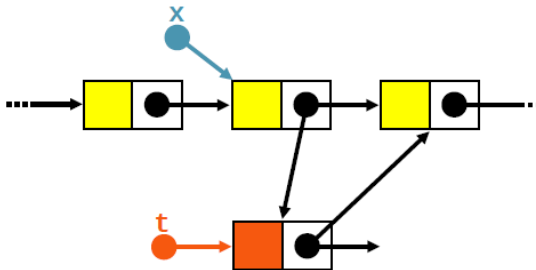


Listas Simplesmente Ligadas

Exemplo - Inserção na Lista

- Inserção na lista de um elemento $*t$ depois de $*x$

```
t->next = x->next;  
x->next = t;
```

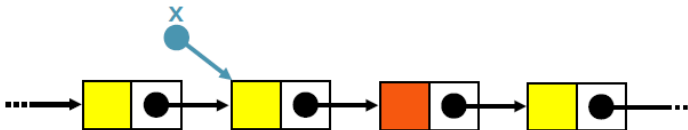


Listas Simplesmente Ligadas

Exemplo - Inserção na Lista

- Inserção na lista de um elemento ***t** depois de ***x**

```
t->next = x->next;  
x->next = t;
```

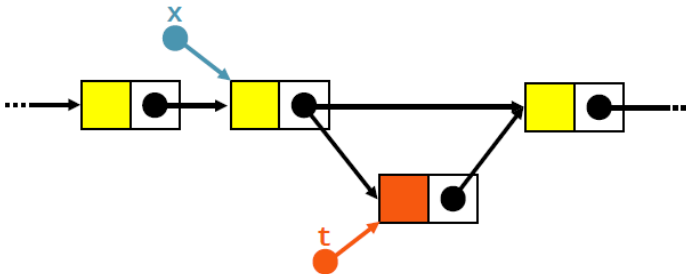


Listas Simplesmente Ligadas

Exemplo - Remoção na Lista

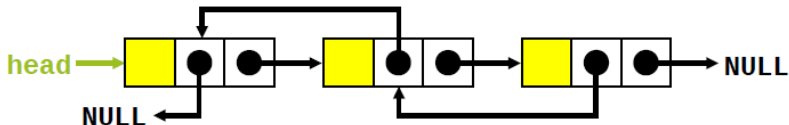
- Remoção do elemento depois de **x** da Lista

```
t = x->next;  
x->next = t->next;  
free(t);
```



Listas Duplamente Ligadas (Listas Biligadas)

- Conjunto de nós



- Cada nó contém
 - Informação útil
 - Ponteiro para o próximo nó e para o nó anterior

```
struct node {  
    Item item;  
    struct node *prev, *next;  
};
```


Estruturas de Dados Dinâmicas

Visualização das estruturas de dados: <https://visualgo.net/en/list>



UNIVERSIDADE
TÉCNICA DO
ATLÂNTICO

CAMPUS
DO MAR